E4 Week 3: Frontend connected to the live backend

Goal

A question typed in the UI reaches the local backend service and the answer renders in the UI.

What I did

Connected our Open WebUI frontend to E1's runtime (`docqa-runtime`, Otto) through its OpenAI-compatible, loopback-only gateway.

1. Started E1's runtime with `start.cmd`. It runs the stub backend automatically when no llama.cpp binary or model is present.
2. In Open WebUI (`http://localhost:3000`): Settings → Admin → Connections → added an OpenAI connection with URL `http://127.0.0.1:8080/v1`.
3. E1's model (`stub-1b-instruct-q4_k_m`) appeared in the model picker.
4. Asked "What is this document about?" in Open WebUI. The request reached the runtime and the response streamed back into the chat.

Health / service status

The runtime reports its state at `GET http://127.0.0.1:8080/health`: `status: ok`, `ready: true`, `loopback_only: true`, `offline_mode: true`. Open WebUI checks the connection and shows "Server connection verified" when the backend is reachable. So backend health is visible on both the backend and the frontend.

Result

Check	Status
Question from the UI reaches the local service	Done
Response renders in the chat	Done
Backend health visible	Done
Everything stays on loopback (127.0.0.1)	Done

Notes

This ran on E1's stub backend, so answers are labeled SIMULATED and carry no meaning. Swapping in the real model only means adding the model file to E1's runtime. No frontend changes are needed.

Next

Library flow (Week 4): choose files, show the validation result, and show errors in the UI.